

Java Generics

Generics allow you to write classes, interfaces, and methods that work with different data types, without having to specify the exact type in advance.

This makes your code more flexible, reusable, and type-safe.

Generic Class

```
class Box<T> {  
  
    T value;  
  
    void setValue(T value) {  
        this.value = value;  
    }  
  
    T getValue() {  
        return value;  
    }  
}  
  
public class GenericClassDemo {  
    public static void main(String[] args) {  
  
        Box<Integer> b1 = new Box<>();  
        b1.setValue(100);  
  
        Box<String> b2 = new Box<>();  
        b2.setValue("Java");  
  
        System.out.println(b1.getValue());  
        System.out.println(b2.getValue());  
    }  
}  
100  
Java
```

Generic Method

```
public class GenericMethodDemo {  
  
    public static <T> void print(T data) {  
        System.out.println(data);  
    }  
  
    public static void main(String[] args) {  
        print(100);  
        print("Java");  
        print(25.5);  
    }  
}
```

```
}  
}  
100  
Java  
25.5
```

Multiple Type Parameters

```
class Pair<K, V> {  
  
    K key;  
    V value;  
  
    Pair(K key, V value) {  
        this.key = key;  
        this.value = value;  
    }  
    void display() {  
        System.out.println(key + " : " + value);  
    }  
}  
  
public class PairDemo {  
    public static void main(String[] args) {  
        Pair<Integer, String> p = new Pair<>(101, "Java");  
        p.display();  
    }  
}  
101 : Java
```